



MONASH
University

Forecasting Airbnb Prices in Mornington

Zehao Gao (31510736)

Antonio Arena (31475167)

Duy Le (32458495)

1. Model

This analysis seeks to compare three models for the purposes of forecasting the average nightly price of an Airbnb in Mornington, Victoria.

1.1 Benchmark Model: Random Forest

A Random Forest model was selected as the benchmark model. The selection process to arrive at this conclusion is detailed in Section 2.3.

In a Random Forest model, the hyperparameters available are *mtry*, *max_depth*, *min_samples_split*, and *min_samples_leaf*. *ntree* specifies the number of decision trees to be included in the ensemble method, and *Mtry* controls the maximum number of features considered for splitting at each tree node. Of the available hyperparameters, only *ntree* and *mtry* were tuned as they are the parameters that are most likely to affect the models' performance.

ntree and *mtry* were tuned using grid search to find the optimal values with the lowest Root Mean Squared Error (RMSE) based on cross-validation. A repeated cross-validation approach was taken, whereby each cross-validation has 10 folds and the process was repeated 3 times. It must be noted that increasing the value of *ntree* generally increases the complexity of the random forest model. Higher *ntree* values would make the model more flexible and robust. If the number of trees is too high, it can also lead to overfitting in the model, increase computational time and potentially reduce interpretability. If the value of *ntree* is too low, the random forest model may not capture the patterns in the data effectively.

Moreover, increases in *mtry* would increase the complexity of the algorithm as well. There would be more variables to be considered at each split. Higher values tend to improve the models' predictive performance, but may also cause the model to underfit the data. Alternatively, lower values result in more correlated trees, increasing the likelihood of overfitting.

1.2 Neural Network 1: 'Tutorial' Model

NN1 applies a feed-forward neural network with 2 hidden layers and a dense output layer. The model utilises the ReLU activation function for all hidden layers due to its efficiency advantages over other options. The linear activation function is used in the output as the model is predicting average price, a continuous variable. The first hidden layer has 128 units, and the second has 64. Dropout rates of 0.4 and 0.3 are applied to the first and second hidden layers respectively. These parameters are chosen based on commonly used values, use of 'rules of thumb' and some intuitive exploration by variation of a single parameter at a time, as evidenced in Chollet and Allaire (2018).

Of note, decreasing the batch size significantly improved the fit. This is likely because the weights were updated a larger number of times for a given number of epochs. A batch size of 16 provided a good balance between efficiency and performance.

Early stopping was implemented to limit possible overfitting and improve computational efficiency. Despite this, the testing and validation loss do appear to converge.

It is worth noting that a grid search was performed to optimise a single layer NN, however the results indicated that this architecture was not appropriate for the problem, as evidenced by the higher loss values in Figure 1 below. This method was therefore terminated in favour of the aforementioned approach.

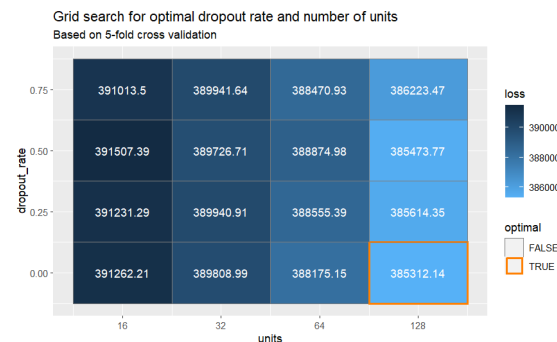


Figure 1: Grid Search Output

1.3 Neural Network 2: Long Short-term Memory (LSTM) Model

For the second neural network model, Long Short-term Memory (LSTM) architecture is chosen due to its popular usage on time series data. LSTM is an architecture based on recurrent neural networks (RNN) and addresses the weaknesses of RNN: vanishing gradient and its limitation on remembering long sequences. LSTM introduces four components, including: Cell state, Forget gate, Input gate, Output gate; and these components allow the layer to determine which less important information can be forgotten at Forget gate.

Data input into this neural network model only uses the average price, with a differencing process on the average price to address the non-stationary factor of the time series. A lag of 1 is also applied on the average price and used as the predictor for the LSTM model instead.

LSTM model is implemented identically to the first neural network model with changes to the input and hidden layer and batch size of 1 is chosen for this. Another parameter to consider was stateful, which allows the internal states to be reused for the samples on the next batch (Richard, 2018). With the LSTM model, a network state reset is also done between each epoch to prevent irrelevant information into the next training epoch, helping the model to adapt and generalise to new patterns in the data.

The activation used is the default tanh function along with default sigmoid function on recurrent activation step, hence the data was normalised to range of $[-1; 1]$ to be used for training. Adam optimizer was used for model training, with a learning rate of 0.02. No further optimization for parameters was done, with results discussed in section 3.3.

2. Experiment

2.1 Data

This analysis was conducted using data from Inside Airbnb (*Get The Data*, n.d.). The data set of interest is 'calendar.csv.gz', which contains nightly prices for 4142 Airbnb's in Mornington from the period of 13/09/2022 to 13/9/2023. A nightly average was taken over each of these listings to generate a single time series of average nightly prices, as shown in Figure X.

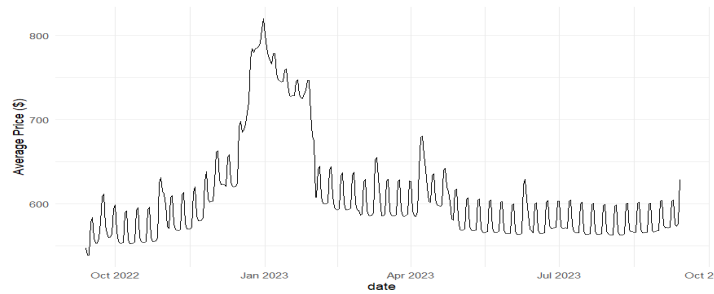


Figure 2: Average Price of Airbnb's in Mornington

To prepare the data for use in the Neural Networks, dummy variables were created for each day of the week, each season, and for particular holiday dates such as Christmas and New Years Day.

2.2 Evaluation Metrics

The primary evaluation metric for all models was RMSE. This metric is appropriate for continuous response variables such as average price. Moreover, RMSE is advantageous to other metrics such as Mean Absolute Error as it penalises large errors more strongly.

2.3 Benchmark Selection

ARIMA, XGBoost and Random Forest were all considered to be the benchmark model for this forecasting problem. Ultimately, the Random Forest was selected due to its advantages over the other two models. Firstly, ARIMA models are most effective to forecast data with trends, however they are unable to handle seasonal data (Brownlee, 2019). Indeed, this was not appropriate as the Airbnb data exhibited strong seasonality, with rises in Summer and falls in Autumn. Moreover, the Random Forest model is superior to XGBoost from a computational sense as it can be fitted more quickly and with less parameter tuning.

Random Forests are an ensemble learning technique that combines the predictions of multiple decision trees to give a robust and accurate result. Random forest models can handle both numeric and categorical data, making them feasible for forecasting Airbnb prices. Random forests tend to be resistant to overfitting, meaning the predictions should be good for unseen data. Moreover, Random forest models are time-efficient as data does not have to be scaled.

3. Results and Discussion

3.1 Random Forest

The results from grid search with cross-validation shows that the optimal values for *mtry* and *ntree* to be 3 and 1000 respectively. The test RMSE for the tuned model is 29.94, with predictions on the test set shown in Figure 3. Predictive performance is fairly well, it captures the weekly seasonality and predicts quite accurate results from the period of March to June. There are deviations in February and June, but it was deemed acceptable given that inputs are limited and could have had more informative features.

The forecast on 295 days in the future which covers the summer period appears to be good, as it captures both the weekly seasonality shown in the data and the rise in average price in summer and fall in autumn.

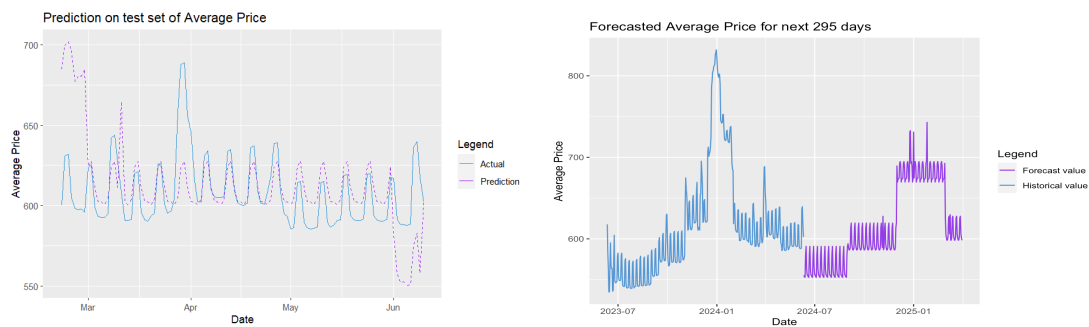


Figure 3: Random Forest Test Set Prediction and Forecasting

3.2 Neural Network 1: 'Tutorial' Model

Considering the plot of test and validation loss against number of epochs shown in Figure 4, it is apparent that despite applying dropout and early stopping, the model does tend to overfit the training data. This may be a consequence of having too many units and otherwise having poor architecture for the learning problem, however this neural network performed well relative to others tested in the optimisation phase.

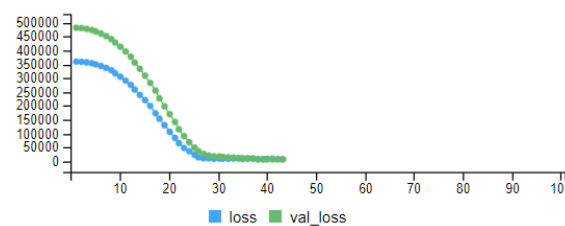


Figure 4: Loss Function vs Epoch

The performance of this model is quite poor, with a test RMSE of 152.78, the highest of all models. As shown in Figure 5, the model massively underestimated the average price from March through to June. This occurs as the training data set contains no observations in these months, thus the model never receives active inputs from these indicators.

The forecast produced by the model is clearly unreasonable. As shown in Figure 5, the forecast consistently underestimates prices for all dates. Moreover, instead of predicting a large peak over the summer period, it predicts a strong crash in average prices. This forecast does not seem reasonable, and it is not recommended to use a model of this format to forecast average prices of Airbnb's in Mornington.

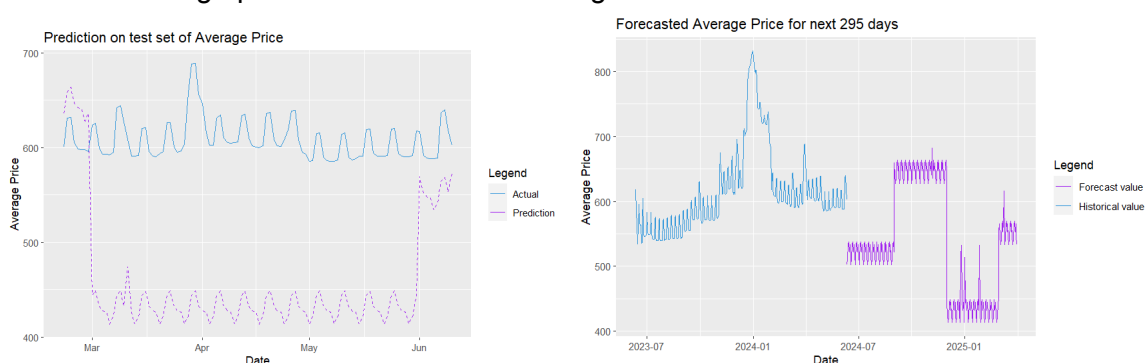


Figure 5: Tutorial Model Test Set Prediction and Forecasting

3.3 Neural Network 2: Long Short-term Memory (LSTM) Model

During the construction of the first two models, the properties of the time series were largely ignored in favour of producing functional models. However, for the LSTM model, the data input was significantly improved by applying differencing to the time series to address the non-stationarity. This enabled the LSTM model to have a test RMSE of 4.89. It must be noted that this does decrease direct comparability between the LSTM model and the other models.

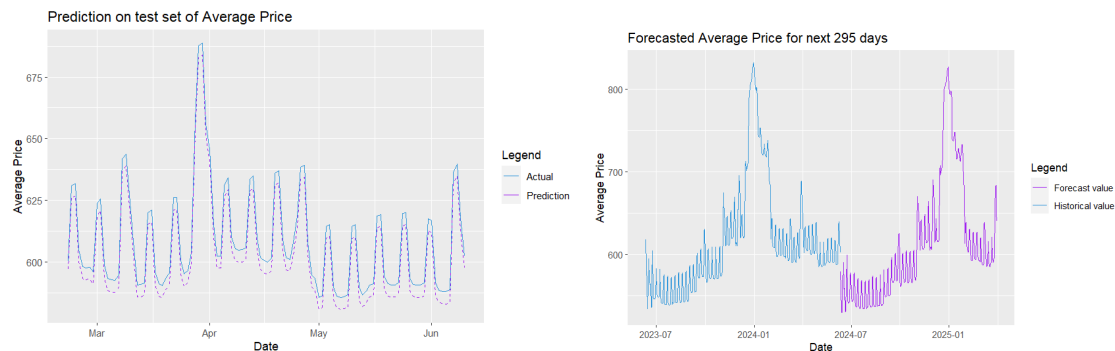


Figure 7: LSTM Test Set Prediction and Forecasting

LSTM outperforms the benchmark model and the deep neural network in terms of RMSE and its ability to capture the pattern of time series without additional features. The performance on the test set suggests that this model is suitable for forecasting average nightly prices during summer period. However, the forecasted average price for next 295 days using the whole 1-year data, indicates signs of overfitting as the forecast closely matches original data. As yet, there is no future data available to verify this.

LSTM models can also take multivariate time series, which includes using other features as inputs. Other features of a house listing are not captured when working with this univariate time series, hence it is expected that the performance of this model could be improved by including other features of listings, such as the number of bedrooms.

4. Conclusion

Throughout this analysis, three models were developed to forecast average nightly prices of Airbnbs in Mornington. A Random Forest model was used as a benchmark, with some tuning to parameters. It produced a test RMSE of 29.94, and a forecast that captures the general increase over summer, but underestimates magnitude. The simple feed-forward NN, produced the highest test RMSE of 152.78 and a forecast that captures weekly seasonality in the data, but does not capture the summer price increase, and underestimates prices. Finally, an LSTM Neural Network was constructed which resulted in a very low test RMSE of 4.89. The forecast from this model appears to capture all features of the original data, however it is uncertain whether it will generalise well to another time series of Airbnb prices.

It is evident that with tuning, tree-based methods can provide effective forecasts for time series data. On the other hand, the simple neural network demonstrated that with this method of dummy-variable input, it was unable to capture most of the important patterns in the data, and thus should not be used in practice for forecasting Airbnb prices. Ultimately, the LSTM model is the most appropriate to be used for forecasting Airbnb prices, however it would benefit from tuning to prevent overfitting and implementation of more predictors in a multivariate time series.

References:

Brownlee, J. (2019). A gentle introduction to SARIMA for time series forecasting in Python. *MachineLearningMastery.com*.

<https://machinelearningmastery.com/sarima-for-time-series-forecasting-in-python/#:~:text=Autoregressive%20Integrated%20Moving%20Average%2C%20or,series%20with%20a%20seasonal%20component>.

Chollet, F., & Allaire, J. (2018). Deep Learning with R. In *Manning Publications Co. eBooks*.
<https://dl.acm.org/citation.cfm?id=3217627>

Get the Data. (n.d.). <http://insideairbnb.com/get-the-data/>

Richard, W. (2018). Time Series Forecasting using LSTM in R. *rwanjohi.rbind.io*.
<http://rwanjohi.rbind.io/2018/04/05/time-series-forecasting-using-lstm-in-r/>